

BEFORE WE START

Pick Your Path

Both paths start at the same QR — scan, then pick on the next page.

RECOMMENDED — PRIMARY

Browser path (CodeKloud)

No local installs. Cluster + terminal in your browser. Recommended for most attendees.

LIMITED — 60 SEATS

Terminal path (Accenture EKS)

Real EKS cluster. Requires Claude Code + AWS CLI + kubectl + git installed locally.

Scan to begin



bubbly-harmony-production-574d

.up.railway.app

Same workshop, same methodology, same scorecard. Pick whichever path matches your setup.

The 90-Minute IDP

AI Ate My Implementation. Let's Build a Platform Together and Score What's Left.

Michael Forrester

Accenture • KCD Texas • May 15, 2026 • Room 3



BEFORE WE GO LIVE

Three Ways Teams Use AI

This talk is about one of them. The one where the agent touches production.

1

Inject AI into workflows

Coding agents, artifact generation, internal automation. Bounded blast radius.

2

Inject AI into an application

Chatbots, search, in-product features. Output is the surface.

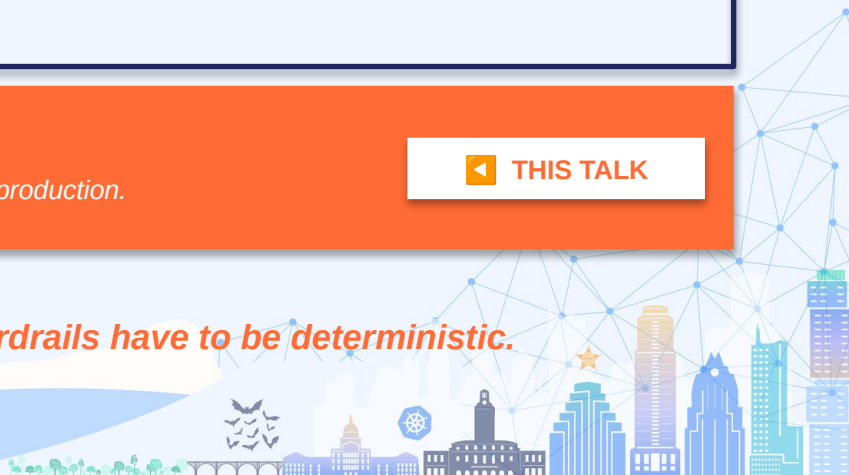
3

AI for systems management

Kubernetes, infrastructure operations, cloud. The agent touches production.

◀ THIS TALK

When the agent touches production, the guardrails have to be deterministic.



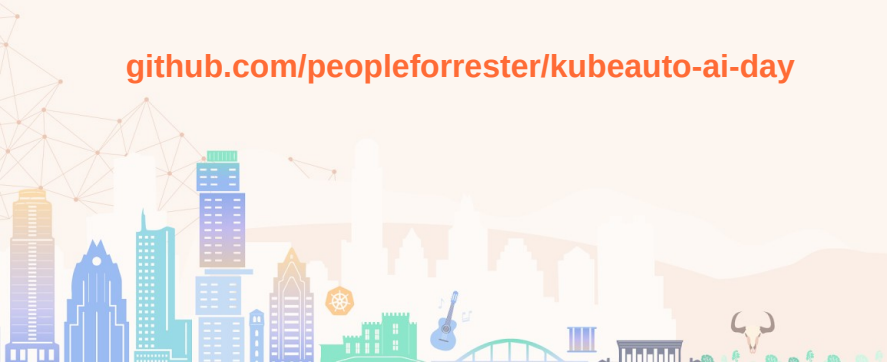
THE HOOK

I've already built this IDP end-to-end with Claude Code.

ArgoCD, Kyverno, Falco, OpenTelemetry, Backstage — the whole stack.

7 phases • 27 components • ~10 hours overnight • Scored on Install / Integration / Usability

github.com/peopleforrester/kubeauto-ai-day

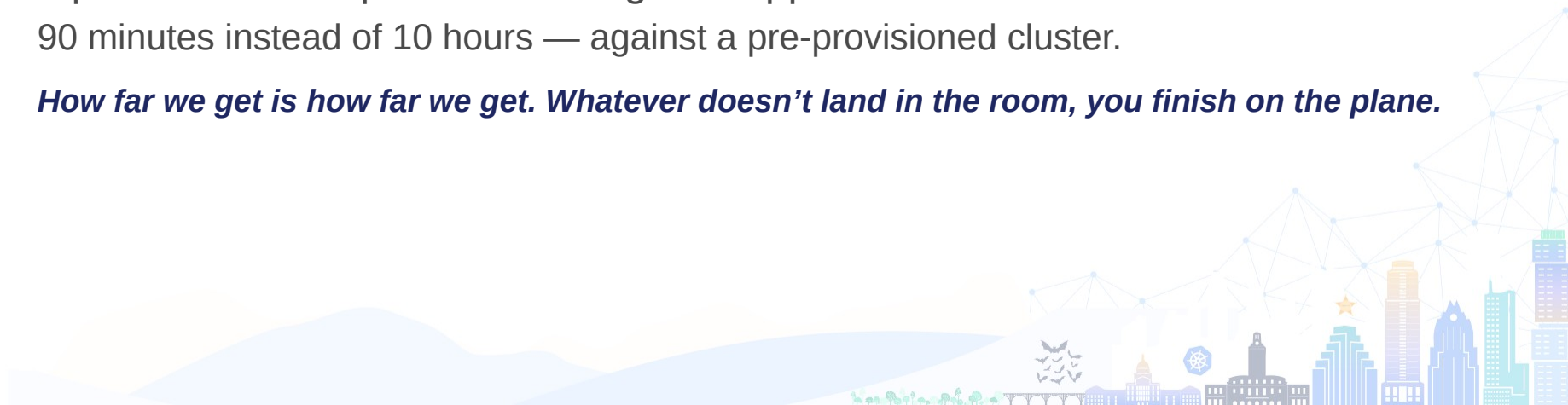


Same build. Real time.

7 phases. 27 components. 21 ArgoCD Applications.

90 minutes instead of 10 hours — against a pre-provisioned cluster.

How far we get is how far we get. Whatever doesn't land in the room, you finish on the plane.



Inside The Repo

Before we go any further — let me show you what's actually in here.

```
KCD_Texas_2026_Workshop/
├── spec/
│   ├── BUILD-SPEC.md           ← the paste
│   └── phases/                 ← 7 phase files
├── .claude/
│   ├── skills/                 ← 6 skill files
│   ├── commands/              ← /build-phase N
│   └── hooks/                  ← stop hook
├── gitops/
│   ├── bootstrap/app-of-apps.yaml
│   └── apps/                   ← 21 Applications
├── tests/                      ← 47 pytest gates
├── scorecard/                  ← 3 dimensions × 7 rows
├── CLAUDE.md                   ← Claude's brief
├── README.md
└── kcd-tx-attendee-playbook.md
```

Pull it up yourself



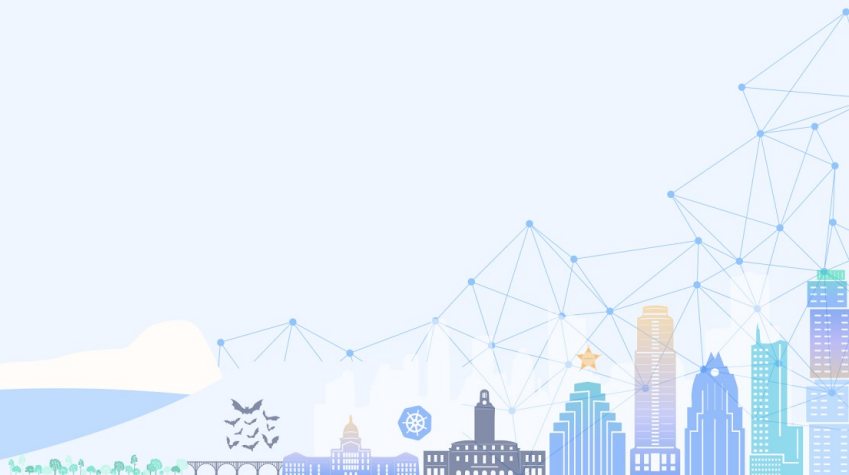
[github.com/peopleforrester
/KCD_Texas_2026_Workshop](https://github.com/peopleforrester/KCD_Texas_2026_Workshop)

I'll walk through the live repo on screen — follow along on your laptop if you're cloning early.

TODAY'S LESSON

Spec-Driven Development with Claude Code

1. I write a spec.
2. I hand it to Claude.
3. Claude generates the platform.
4. I verify with test gates.
5. I score what Claude did, and move on.



THE THREE ARTIFACTS

Spec + Skills + Gates

The Spec

What I want Claude to build.
Phases, manifests, completion
criteria.

*spec/BUILD-SPEC.md — ~120 lines
markdown.*

The Skills

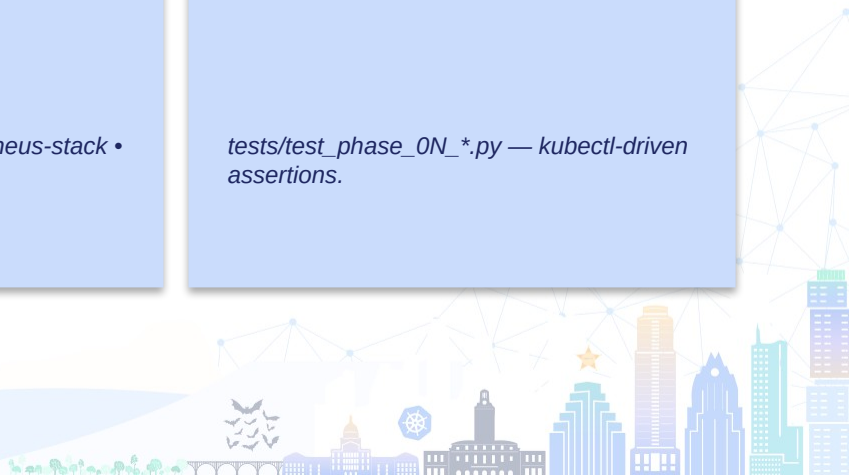
Current-version patterns Claude
needs. One per CNCF project.
Auto-loaded.

*argocd • kyverno • kube-prometheus-stack •
backstage*

The Gates

pytest tests proving a phase
actually worked. Real
infrastructure, no mocks.

tests/test_phase_ON_.py — kubectl-driven
assertions.*



Three Dimensions, Scored Independently

Install

Did Claude generate a manifest that came up healthy?

Pods Running. No rewrites. First try.

Integration

Does it work with the other components?

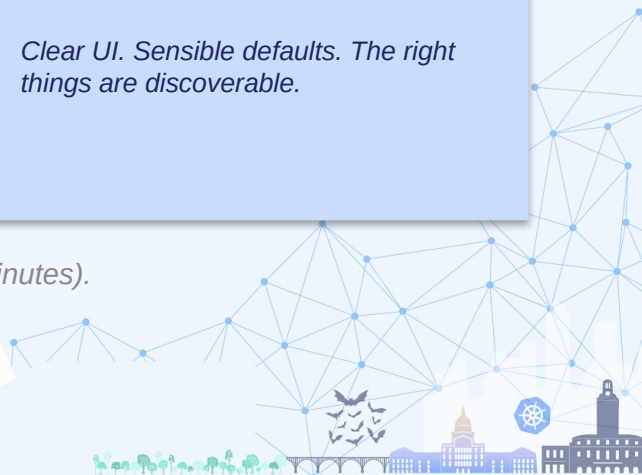
Sync waves, webhooks, scrape, policy collisions. Where humans still earn salary.

Usability

Could a developer drive this Monday morning?

Clear UI. Sensible defaults. The right things are discoverable.

Plus: correction cycles (count) and AI wall-clock time (minutes).



7 Phases. 27 Components. As Far As We Get.

1

Foundation

Assert pre-provisioned cluster, IAM, namespaces (no manifests generated)

2

GitOps

ArgoCD install + app-of-apps → 21 child Applications reconcile in parallel

3

Security

Kyverno + 3 policies, Falco + falcosidekick + falco-talon, ESO, RBAC, NetworkPolicies

4

Observability

kube-prometheus-stack, Grafana, OTel Collector, Loki, Promtail, Tempo, ArgoCD ServiceMonitors

5

Portal

Backstage (with appConfig override + image pin — the trap that defines the phase)

6

Integration

Cross-component: GitOps drift reversion, admission firing during scrape, audit trail

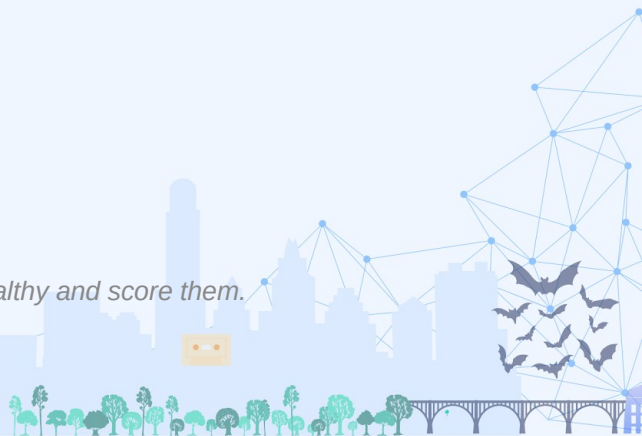
7

Hardening

cert-manager + ClusterIssuers, ResourceQuotas, PDBs

Phase 2 fans out everything. Phases 3–7 wait for components to reach Healthy and score them.

How far we get is how far we get.



NOW WE GO LIVE

Open your laptops.

Whichever path you picked from slide 1, get to a working claude session:

A

BROWSER (KodeKloud)

```
# Already in browser shell
$ kubectl get nodes # nodes Ready
$ git clone <repo-URL>
$ cd KCD_Texas_2026_Workshop
$ claude
> Paste prompt from
  spec/BUILD-SPEC.md (top)

# Claude detects cluster type
# and adapts Phases 1, 3, 7
# to vanilla kubeadm vs EKS.
```

B

TERMINAL (Accenture EKS)

```
# Creds from Railway success page
$ aws configure
$ aws eks update-kubeconfig \
  --name kcd-tx-attendee-NN \
  --region us-east-2
$ kubectl get nodes # nodes Ready
$ git clone <repo-URL>
$ cd KCD_Texas_2026_Workshop
$ claude
> Paste prompt from
  spec/BUILD-SPEC.md (top)
```

I'm on EKS up here; most of you are on KodeKloud. Phases 3 + 7 will look different on your screen vs. mine — that divergence IS the data.

No TAs today — just me. Setup window is the only time I can help individuals.

WHAT YOU'RE ABOUT TO WATCH

The Spec-Driven Loop

1

I paste the spec

spec/BUILD-SPEC.md — single paste, autonomous run.

2

Claude reads the skill

argocd-patterns / kyverno-policies / kube-prometheus-stack / backstage-templates.

3

Claude generates a manifest

~/my-<component>.yaml — diffed against the pre-committed gitops/ ground truth on the projector.

4

We apply + run the gate

kubectl apply, then pytest tests/test_phase_ON_*.py -v.

5

Promise on green

<promise>PHASE_N_DONE</promise> only when ALL tests pass. Otherwise PHASE_N_FAILED.

6

Score and continue

Install / Integration / Usability on the live scorecard. Then “continue” and Claude moves to Phase N+1.



WHAT CLAUDE KNOWS BEFORE WE START

Skills Auto-Loaded from `.claude/skills/`

`argocd-patterns.md`

Chart 9.x → ArgoCD 3.x. Annotation-based tracking. SSA on CRD-heavy charts.

Prevents: timeout.reconciliation at wrong path; legacy 2.x resourceTrackingMethod.

`kyverno-policies.md`

Webhook-level namespace exclusion via chart values. background: false on workshop policies.

Prevents: per-policy excludes; case-mismatched validationFailureAction.

`falco-rules.md`

CRITICAL-tagged custom rules for credential access. Falcosidekick → Prometheus wiring.

Prevents: noisy default rules; missing falcosidekick metrics; eBPF probe mismatch.

`kube-prometheus-stack.md`

ServerSideApply mandatory. alertmanager disabled. 2h retention. Pre-defined Helm values.

Prevents: CRD-annotation overflow; default 10d retention burning disk; alertmanager Pending.

`otel-wiring.md`

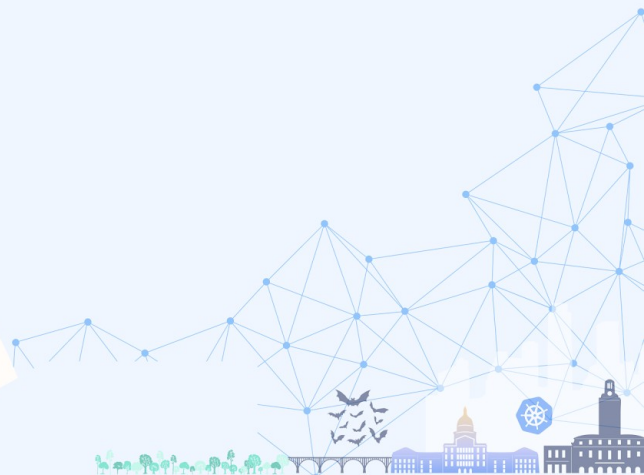
memory_limiter MUST be first processor in every pipeline.

Prevents: collector OOM under load; cascading backpressure to receivers.

`backstage-templates.md`

Image pinned to ghcr.io/backstage/backstage:1.30.2 + appConfig kubernetes-plugin override.

Prevents: no-default-image trap; Kubernetes-plugin init crash; wrong backend-system era.



WHAT'S RUNNING UNDERNEATH

The Guard Rails You Don't See

Pre-commit hooks

gitleaks • yamllint • kubeconform
• helm-lint • shellcheck

*Runs on every commit to staging. Bad
YAML / leaked secrets never reach the
branch.*

pytest gates

tests/test_phase_ON_*.py against
the real cluster. ALL must pass.

*Promise <PHASE_N_DONE/> only fires
when pytest exits 0. No faked passes.*

GitOps reconciliation

ArgoCD reads main. Pre-commit
blocks bad commits. Promotion
staging → main is local fast-forward.

*Single-maintainer rigor: no CI/CD, no
remote gates — the local hooks ARE the
gates.*

Same rigor as kubeauto-ai-day, scoped to local-only for a single-maintainer workshop repo.



What the Scorecard Showed

Install

↑ **High**

AI generates manifests well. First-try success is real.

Integration

→ **Middling**

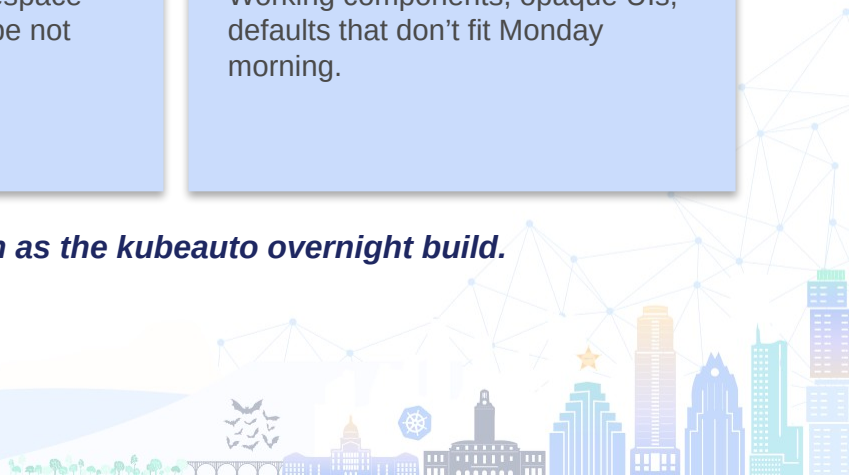
Sync waves missed, namespace exclusions forgotten, scrape not wired.

Usability

↓ **Low**

Working components, opaque UIs, defaults that don't fit Monday morning.

Install » Integration » Usability — same pattern as the kubeauto overnight build.



THE TAKEAWAY

The implementation layer didn't disappear. *It compressed.*

AI handles the YAML and Helm values faster than I can.

What's left is sync wave ordering, namespace exclusions, policy collisions, image configs that don't match the chart — the judgment calls that turn CNCF tools into a platform someone can actually use Monday morning.

That's the engineering work that didn't go anywhere.

TAKE HOME

Methodology + Scorecard + Reference Build

KEEP BUILDING — LABS STAY UP AFTER THE WORKSHOP

Accenture EKS clusters: live until end of day Friday • KodeKloud labs: available through Monday



Workshop Repo

github.com/peopleforrester/

KCD_Texas_2026_Workshop

Spec, skills, gates, scorecard



Reference Build

github.com/peopleforrester/

kubeauto-ai-day

7 phases, 27 components, full scorecard



Agentic Covenants

github.com/peopleforrester/

agentic-covenants

Governance framework for autonomous agents



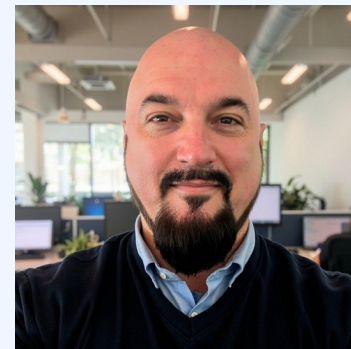
Thank you.

Questions?

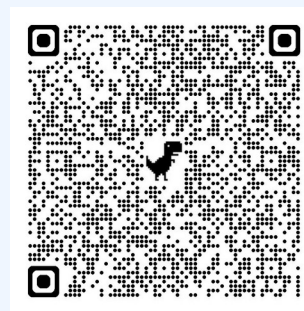
Michael Forrester

AI Workplace Transformation Specialist — Accenture

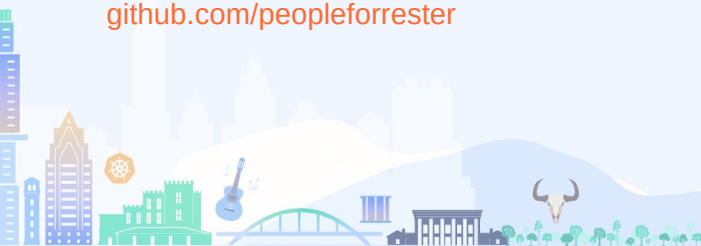
github.com/peopleforrester



Michael Forrester



Scan: LinkedIn



The Hooks — Deterministic Enforcement

Claude Code Stop Hook

`.claude/hooks/cc-stop-deterministic.sh`

```
if [ ! -f .build-active ]; then
  approve # hook inactive
fi

if grep PHASE_[1-7]_DONE; then
  approve # phase complete
fi

block # promise not emitted
```

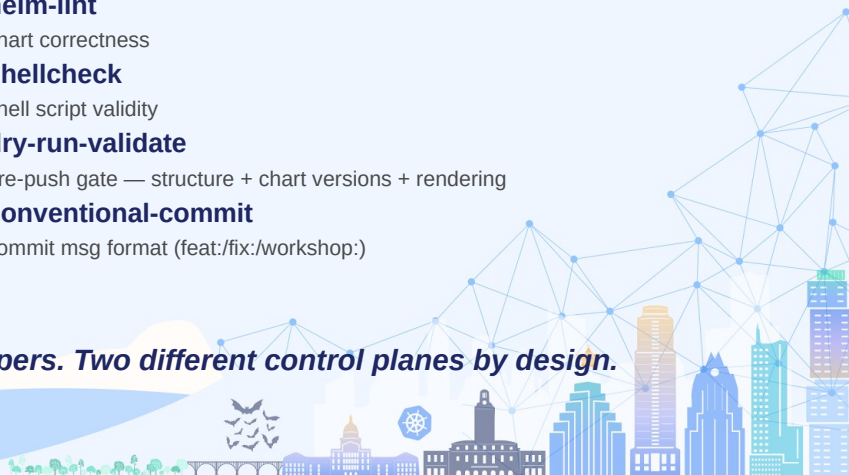
Stops Claude from "approximately done." Phase completion requires the literal `<promise>PHASE_N_DONE</promise>` tag. No promise → hook blocks the Stop event → Claude must keep iterating until pytest passes.

Pre-Commit Gates (local enforcement)

`.pre-commit-config.yaml` — cannot be bypassed by Claude

- gitleaks**
secret scan — runs first, blocks if hit
- yamllint**
YAML syntax + style
- kubeconform**
K8s schema validation against real CRDs
- helm-lint**
chart correctness
- shellcheck**
shell script validity
- dry-run-validate**
pre-push gate — structure + chart versions + rendering
- conventional-commit**
commit msg format (feat./fix./workshop:)

The agent is the YAML generator. The hooks are the gatekeepers. Two different control planes by design.



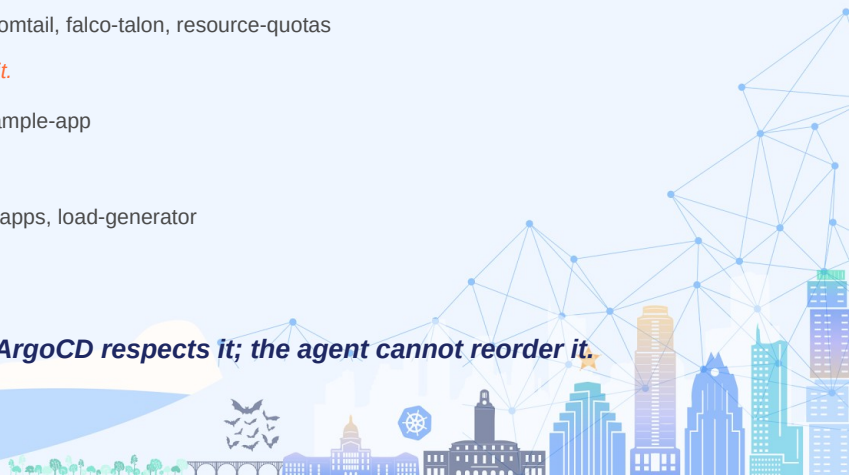
APPENDIX B

The Wave — Sync Wave Architecture

32 Applications. 11 waves. One DAG that ArgoCD walks bottom-up.

-10	Foundation	namespaces <i>All workshop namespaces exist before anything tries to deploy into one.</i>
-5 to -2	Security pre-empt	kyverno → kyverno-policies + RBAC + NetPol + ESO + Falco family <i>Admission controller + policy enforcement is in place BEFORE workloads arrive.</i>
1 to 2	Platform services	cert-manager, kube-prometheus-stack, OTel, ServiceMonitors, cert-manager-issuers <i>TLS infra + metrics scraping running before anything they monitor exists.</i>
3 to 4	Observability + remedy	grafana-dashboards, loki, tempo, promtail, falco-talon, resource-quotas <i>Logs/traces/dashboards/auto-remediation — visibility before workloads need it.</i>
5	Portal + sample	backstage, backstage-resources, sample-app <i>Developer surface. Backstage and sample workload run in parallel.</i>
7 to 8	Demo workloads	ecom (api/frontend/worker), 5 party-apps, load-generator <i>Real workloads to drive the policy/scrape/log/trace pipelines.</i>

Wave order encodes architectural truth: what depends on what. ArgoCD respects it; the agent cannot reorder it.



The Sink — Falco → Falcosidekick → Falco-Talon

Runtime detection → multi-sink fanout → automated response. Three components, one pipeline.



Detection separated from fanout separated from response. Each piece swappable without redesign.

APPENDIX D

The Skills — What Claude Reads Before Generating

.claude/skills/ — auto-loaded when claude starts in this repo. Encodes traps training data doesn't.

argocd-patterns.md

argo-cd 9.x → ArgoCD 3.x

configs.cm path for timeout (not configs.params). Annotation-tracking default in 3.x. Sync wave conventions — always pre-set, never let agent invent.

kyverno-policies.md

kyverno chart 3.8.0

Webhook namespaceSelector as map, not list. ServerSideApply=true on the policies App. validationFailureAction case-sensitive (Enforce not enforce).

kube-prometheus-stack.md

kps chart 84.5.0

SSA=true MANDATORY — CRDs blow past annotation size limit otherwise. Alertmanager disabled for pacing. 2h retention.

backstage-templates.md

backstage chart 2.7.0

No default image — pin to ghcr.io/backstage/backstage:1.30.2. AppConfig kubernetes plugin override prevents init crash. Roadiehq image DOES NOT EXIST.

falco-rules.md

falco + sidekick + talon

CRITICAL-tagged custom rules. Falcosidekick → Prometheus scrape AND falco-talon webhook in same config. eBPF probe match for kernel ver.

otel-wiring.md

opentelemetry-collector

memory_limiter MUST be first processor in every pipeline. Without it: collector OOM under load, cascading backpressure to receivers. Subtle, expensive.

Training data is stale on these specific charts. Skills are how Claude generates current patterns instead of remembering deprecated ones.



APPENDIX E

The Scorecard — Three Dimensions, Independent

Score each phase across three dimensions — independently. The variance is the talk.

Install

Did the manifest, after applying, bring the phase's components up healthy? First try, no rewrites?

10 — First-try success. Pods Running, Applications Synced + Healthy.

4 — Three correction cycles, registry workaround, version archaeology.

Integration

Do the components do their JOB — in concert with everything around them? Not just "Running."

10 — Policies fire on real pods, dashboards populate, traces flow.

4 — Pods Running, feature does not actually work end-to-end.

Usability

Could a developer on your team drive this on Monday morning?

10 — Production-ready, junior engineer ships through it.

4 — Installed but unship-able.

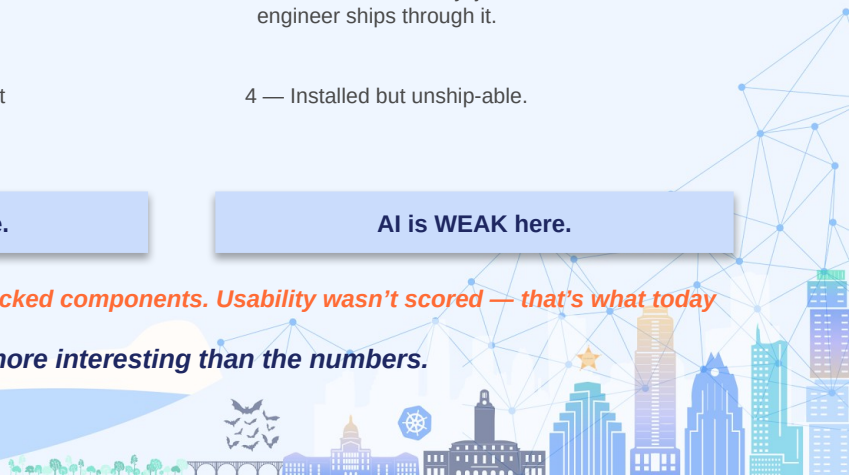
AI is GREAT here.

AI is MIDDLING here.

AI is WEAK here.

Kubeauto overnight baseline: Install avg ≈ 8.1 , Integration avg ≈ 8.5 across the 7 tracked components. Usability wasn't scored — that's what today adds.

Whatever the workshop scores live, the pattern is more interesting than the numbers.



Autonomous Execution — Paste → Hook → Promise

One paste runs all 7 phases. The hook + the promise contract make it work.

STEP 1

I paste the spec

Single paste from
spec/BUILD-SPEC.md's
"How Claude executes
this spec" block.

*Spec defines all 7 phases,
scoring dimensions, gate
format, stack pins.*

STEP 2

Hook enforces the contract

.claude/hooks/cc-stop-
deterministic.sh refuses
the Stop event without a
<promise>PHASE_N_DONE</promise>.

*If pytest didn't pass,
the promise tag never
fires, Claude is forced
to keep iterating.*

STEP 3

Promise lands or fails honestly

pytest exit 0 → emit
<promise>PHASE_N_DONE</promise>
→ hook releases
→ advance to phase N+1.

*pytest fails after diagnostic →
emit PHASE_N_FAILED.
Honest failures > faked passes.*

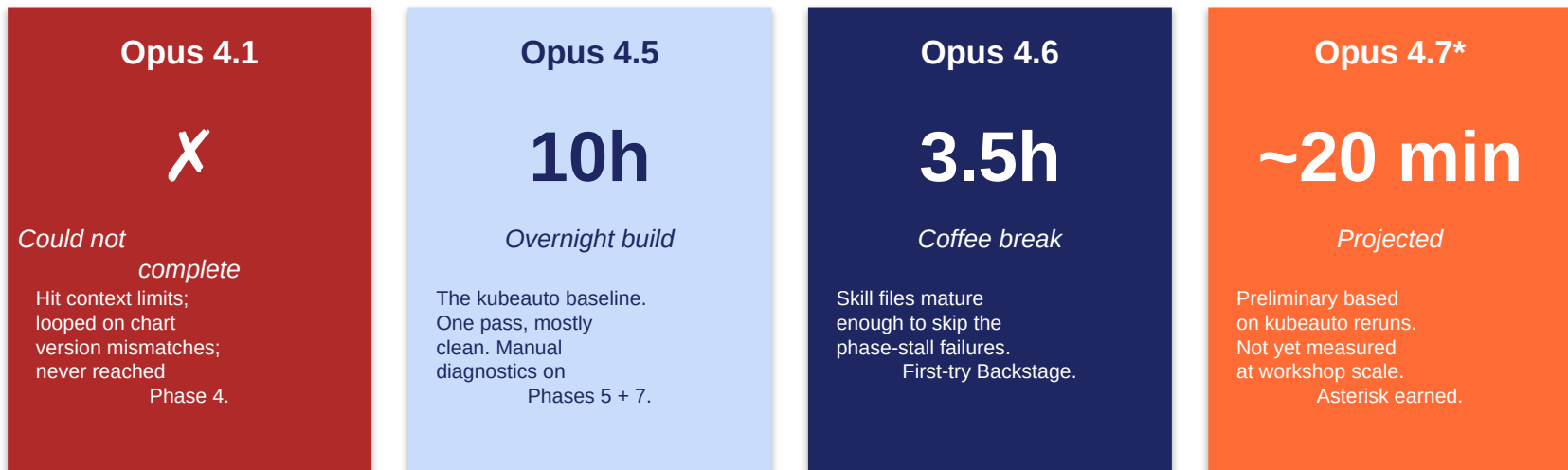
Stuck or fell behind? /build-phase N reruns just one phase — catch-up path with the same hook + promise contract.

Same gate, same contract, single-phase scope. Used when the autonomous loop gets stuck or a student catches up mid-talk.



The Acceleration — Same Spec, Different Agent

Same 7-phase build. Same scorecard. Different Claude. Watch the timeline collapse.



The methodology is stable. The agent isn't.

Write the spec once. Every model that comes after compounds the investment.

** 4.7 estimate is from internal kubeauto reruns, not a full workshop run. Don't treat as benchmark data.*

What Kubeauto Taught Us — AI Is Asymmetric

Observed across multiple live workshop builds, not direct from the kubeauto data. Build for the shape — not against it.

INSTALL

7–9

AI is GREAT here

Templating Helm values, naming resources, generating manifests. First-try success is common across nearly every component.

INTEGRATION

5–7

AI is MIDDLING here

Cross-component wiring, webhook scoping, sync wave coupling, scrape selectors. Components install, but don't actually do their job in concert.

USABILITY

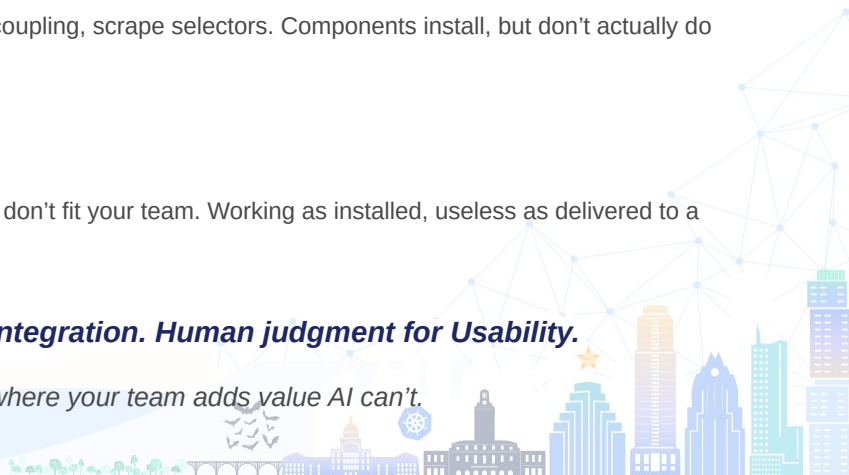
4–6

AI is WEAK here

Empty Backstage catalogs, stub templates, defaults that don't fit your team. Working as installed, useless as delivered to a developer.

The heuristic: skill files for Install. Pre-commit hooks for Integration. Human judgment for Usability.

Match your investment to where AI is weakest. That's where your team adds value AI can't.



What Kubeauto Actually Showed

Published data from the overnight build (Feb 2026, EKS 1.34, Opus 4.x). 27 components scored.

3h 10m*AI time*

Total wall-clock across
27 components

12h 5m*Manual est.*

What it would have taken
by hand

73.8 %*Toil reduction*

725 → 190 min net
reduction

25*Corrections*

Across 27 components
(≈11 zero-correction)

Per-component scores (7 of 27 tracked on Install/Integration)

Component	Install	Integ.	Cycles
ArgoCD install	8	9	0
App-of-apps	9	9	1
Kyverno install	7	—	3
Kyverno policies	9	9	1
kube-prometheus-stack	8	8	1
Grafana dashboards	9	9	0
Backstage	7	7	1

Avg Install ≈8.1 • Avg Integration ≈8.5 • Usability not scored

Source: github.com/peopleforrester/kubeauto-ai-day — [scorecard/raw-scores.json](#) + [spec/SCORECARD.md](#)

Where Claude struggled (3 cycles each)

EKS Cluster

Module v21.15 renames; AWS provider 6.x; addon chicken-and-egg.

Kyverno install

Skill file webhook wrong (list vs map, chart 3.7.0); needed SSA.

OTel Collector

Chart 0.145.0 breaking change; k8s image missing exporter.